

Approach Document

OCR/Document Parsing Approach

I used **PyMuPDF (fitz)** to extract the document content because the provided **CT-200** manuals were digitally generated PDFs with selectable text rather than scanned images. Since OCR was not required for these documents, extracting text directly was simpler, faster, and avoided OCR-related recognition errors. **PyMuPDF** also preserves page order efficiently and allows me to process the document section by section before reconstructing the hierarchy.

Hierarchy Reconstruction Strategy

The document was reconstructed into a hierarchical tree using numbered headings detected through regular expressions. Each heading was assigned a hierarchy level based on the numbering pattern (for example, 1, 1.1, 1.1.1). While parsing sequentially, the current parent section was tracked so that every section could be linked with its parent. Each node stores its section number, title, body content, hierarchy level, parent reference, and a SHA-256 content hash for later version comparison.

Edge Cases

During parsing I observed that some numbered list items resembled headings because they began with numeric prefixes. A simple regular expression could incorrectly classify these items as section headings. Although the majority of the document followed a consistent numbering format, this demonstrated that relying only on text patterns is insufficient for perfect hierarchy reconstruction. A more robust implementation would additionally use font size, font weight, indentation, or layout information.

Initial Implementation Failure & Rectification

My initial implementation relied entirely on heading numbering. As a result, numbered bullet lists could occasionally be interpreted as section headings. The parser also assumes a relatively consistent heading structure and would not perform well on documents with heavily inconsistent formatting or scanned pages requiring OCR. These **issues** were identified through manual inspection of the parsed hierarchy and by comparing the extracted sections with the original PDF. During version comparison, I noticed that some numbered list items appeared as separate sections, which indicated that the heading detection rule was too broad. This manual validation helped identify parser limitations before finalizing the implementation.

Extraction Quality Changes

I refined the heading parsing logic and improved hierarchy tracking so that parent-child relationships were reconstructed more consistently. Content hashing was also introduced to support version comparison and staleness detection. Due to the project timeline, I intentionally stopped short of implementing layout-aware heading detection and instead documented this limitation.

Decision Log

Q1. What's the one part of this system most likely to silently give wrong results without erroring? How would you catch it?

The parser's heading detection is the most likely component to silently produce incorrect results. If a numbered list item resembles a section heading, the parser may assign it an incorrect position in the hierarchy without raising an error. I would detect this using manual hierarchy validation, parser unit tests for known edge cases, and by comparing extracted structures against the original PDF during development.

Q2. Where did you choose simplicity over correctness because of time, and what would break first if this went to production as-is?

For version matching, I chose a section-number-and-title matching strategy combined with content hashing instead of implementing semantic similarity matching. This simplified the implementation while still supporting document versioning. In production, this approach could incorrectly classify renamed sections as new nodes even if their meaning remained unchanged. A semantic matching approach using embeddings or fuzzy similarity would provide better accuracy.

Q3. Name one input (to your parser, your versioning matcher, or your LLM call) that you did *not* handle, and what your system does when it sees it.

My parser does not handle scanned PDFs that require OCR or documents with highly inconsistent formatting. Since the provided CT-200 manuals contained selectable text, the parser assumes text extraction is available. If a scanned document were provided, the current implementation would either extract little or no meaningful content, and the hierarchy reconstruction would fail without an OCR preprocessing stage.

Flowchart of program

